

```
# Import Libraries
import numpy as np
import pandas as pd
import scipy.stats as stats
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import r2_score, mean_squared_error, mean_absolute_error
from statsmodels.stats.multicomp import pairwise_tukeyhsd
import plotly.express as px
import warnings
warnings.filterwarnings("ignore")

# Define Parameters
np.random.seed(42)
session_levels = ["Short", "Medium", "Long"]
break_levels = ["Short", "Medium", "Long"]
difficulty_levels = ["Low", "Medium", "High"]
time_levels = ["Morning", "Afternoon", "Evening", "Night"]
sessions_per_scenario = 200

# Generate Dataset (21600 rows)
# 3 × 3 × 3 × 4 × 200 = 21,600 rows

data = []
for s in session_levels:
    for b in break_levels:
        for d in difficulty_levels:
            for t in time_levels:
                for i in range(sessions_per_scenario):
                    # Session Length
                    if s == "Short":
                        session_val = np.random.uniform(25, 45)
                    elif s == "Medium":
                        session_val = np.random.uniform(60, 90)
                    else:
                        session_val = np.random.uniform(120, 180)
                    # Break Duration
                    if b == "Short":
                        break_val = np.random.uniform(5, 10)
                    elif b == "Medium":
                        break_val = np.random.uniform(15, 20)
                    else:
                        break_val = np.random.uniform(25, 35)
                    # Difficulty Encoding
                    diff_val = {"Low":1, "Medium":2, "High":3}[d]
                    # Time Effect
                    time_factor = {
                        "Morning": 1.1,
                        "Afternoon": 1.0,
                        "Evening": 0.9,
                        "Night": 0.85
                    }[t]
                    # Fatigue
                    fatigue = 0.02 * session_val * diff_val
                    # Recovery
                    recovery = np.log(1 + break_val)
                    # Performance
                    performance = (100 * time_factor) - fatigue + (8 * recovery)
                    # Error Count
                    error_lambda = max(1, diff_val * session_val / 50)
                    errors = np.random.poisson(error_lambda)
                    data.append([
                        s, session_val,
                        b, break_val,
                        d,
                        t,
                        fatigue,
                        recovery,
                        performance,
                        errors
                    ])
df = pd.DataFrame(data, columns=[
    "Session Category",
    "Session Length",
    "Break Category",
    "Break Duration",
    "Difficulty",
    "Time of Day",
    "Fatigue",
    "Recovery Index",
    "Performance",
    "Error Count"
])
print(df.shape)

# Preprocessing
df_model = df.copy()
df_model = pd.get_dummies(df_model, columns=[
    "Session Category",
    "Break Category",
    "Difficulty",
    "Time of Day"
], drop_first=True)

# Save Dataset
df.to_csv("synthetic_cognitive_dataset.csv", index=False)

# Descriptive Statistics
desc_stats = df.describe()
desc_stats

# KS TEST (Distribution Validation)
ks_results = {}
for col in ["Performance", "Fatigue", "Recovery Index", "Error Count"]:
    stat, p = stats.kstest(df[col], 'norm', args=(df[col].mean(), df[col].std()))
    ks_results[col] = {"KS Statistic": stat, "p-value": p}
ks_table = pd.DataFrame(ks_results).T
ks_table

# ANOVA (Break Duration Effect)
anova = stats.f_oneway(
    df[df["Break Category"]=="Short"]["Performance"],
    df[df["Break Category"]=="Medium"]["Performance"],
    df[df["Break Category"]=="Long"]["Performance"]
)
anova

# Tukey HSD
tukey = pairwise_tukeyhsd(
    endog=df["Performance"],
    groups=df["Break Category"],
    alpha=0.05
)
print(tukey)

# Cohen's d
def cohens_d(x, y):
    return (x.mean() - y.mean()) / np.sqrt((x.std()**2 + y.std()**2)/2)
d_short_long = cohens_d(
    df[df["Break Category"]=="Short"]["Performance"],
    df[df["Break Category"]=="Long"]["Performance"]
)
print("Cohen's d:", d_short_long)

# Grouped Tables
group_table = df.groupby("Break Category")["Performance"].agg(["mean", "std", "min", "max"])
group_table

# Correlation Matrix + Heatmap
corr = df.corr(numeric_only=True)
corr

# Correlation Heatmap
fig = px.imshow(corr,
    text_auto=True,
    title="Figure 4.1: Correlation Heatmap")
fig.show()

# Predictive Model
X = df_model.drop("Performance", axis=1)
y = df_model["Performance"]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Linear Regression
lr = LinearRegression()
lr.fit(X_train, y_train)
lr_pred = lr.predict(X_test)

# Random Forest
rf = RandomForestRegressor(n_estimators=100, random_state=42)
rf.fit(X_train, y_train)
rf_pred = rf.predict(X_test)
print("Linear Regression R2:", r2_score(y_test, lr_pred))
print("Random Forest R2:", r2_score(y_test, rf_pred))

# Visualization
# Box Plot
fig = px.box(df, x="Break Category", y="Performance", color="Break Category")
fig.show()
# Scatter Plot
fig = px.scatter(df, x="Session Length", y="Performance", color="Difficulty")
fig.show()
# Bar Chart (Time of Day)
fig = px.bar(df.groupby("Time of Day")["Performance"].mean().reset_index().sort_values("Performance", ascending=False),
    x="Time of Day", y="Performance", title="Performance by Time of Day")
fig.show()
```